

iscte

INSTITUTO
UNIVERSITÁRIO
DE LISBOA

Data Science Fundamentals

Class 4 – High Performance Python: NumPy, Data Wrangling w/ Pandas 1

MSc. in Computer Science and Business Management

Optional

Professor:

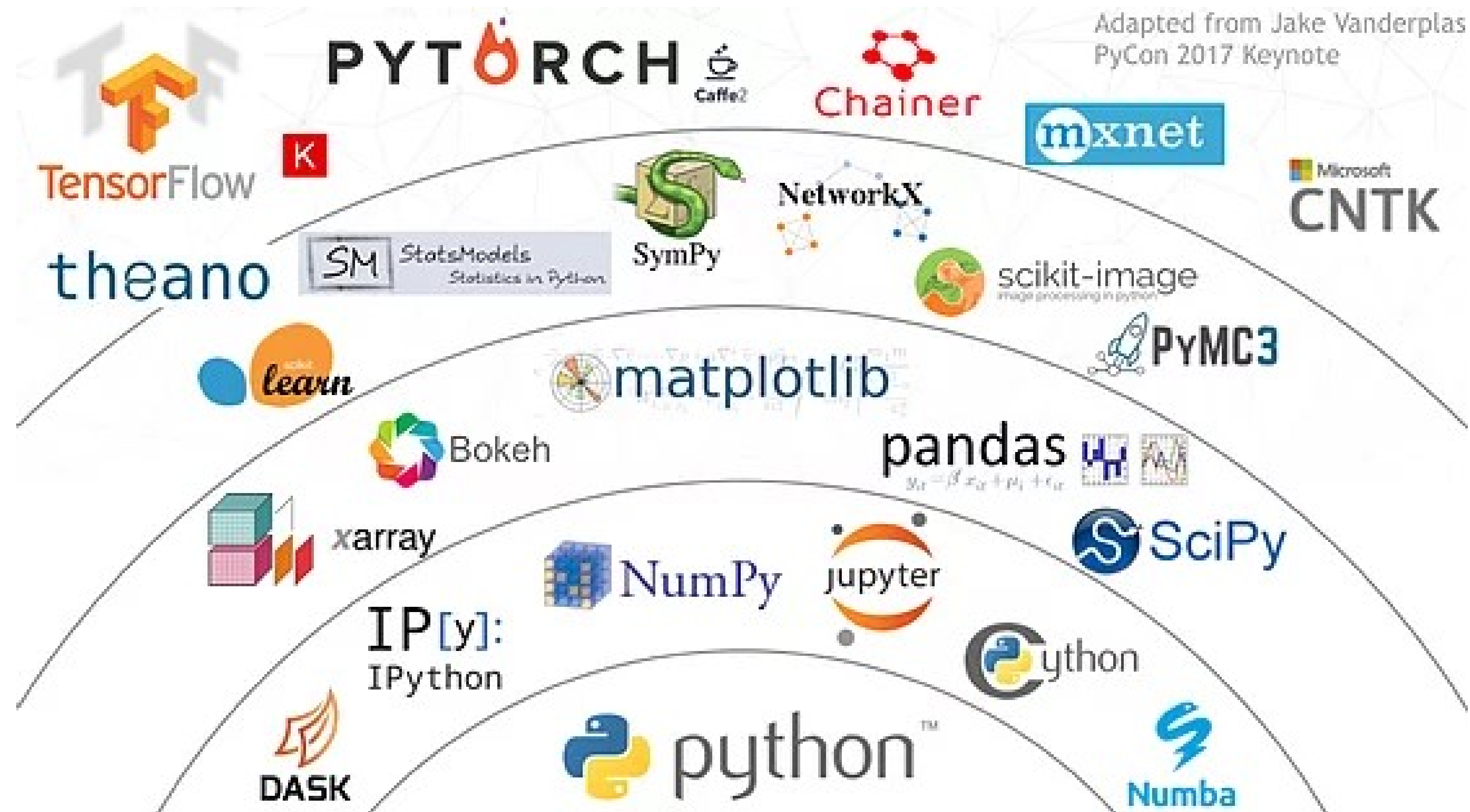
António Raimundo

Year 2022/2023

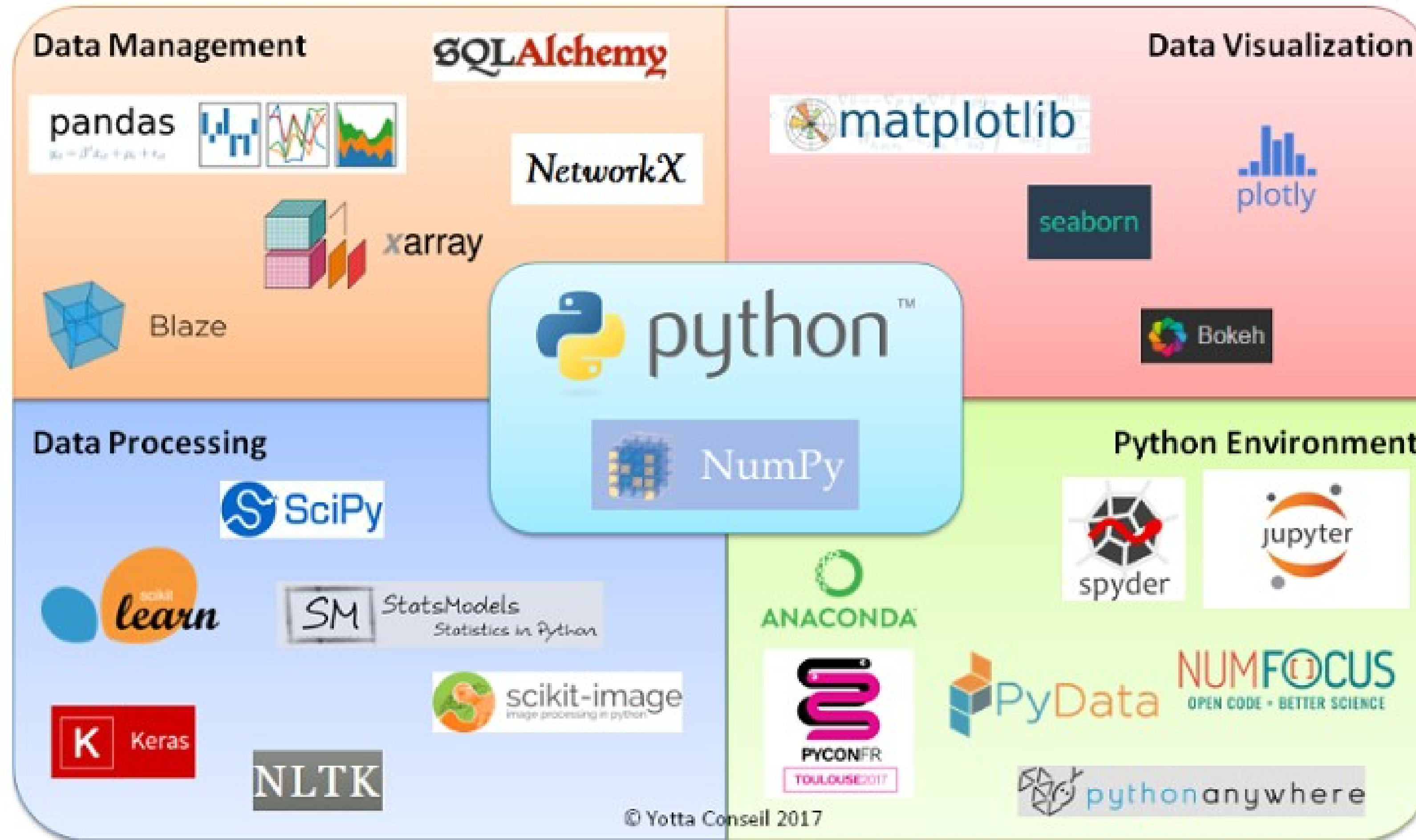
Outline

- **High Performance Python: NumPy**
 - Introduction to Numpy library
- **Data Wrangling with Pandas #1**
 - Introduction to Pandas library
 - Data Loading

Python – Scientific Ecosystem (1)



Python – Scientific Ecosystem (2)



NumPy – What is it?

- **NumPy** – Short for 'Numerical Python'
- It is the open-source base library for **scientific computing** in Python.
- Provides the **data structures, algorithms, and methods** for most **scientific applications** involving **numerical data** in Python.
- Uses **a high-performance data structure** known as **a multi-dimensional array or tensor** (*n-dimensional array*) or ***ndarray***, for the efficient computation of **vectors / matrices / tensors**.
- Allows to perform **linear algebra operations, transforms, random number generation**, and a multitude of other capabilities.
- **Developed in C** (***blazing fast***) and applied in Python, it dramatically improves overall performance.
- Besides its obvious scientific uses, NumPy can also **be used for generic data**, due to its excellent efficiency to handle data of any dimension (*n-dimensional*)



NumPy – Vectors, Matrices, and Tensors?

Scalar Vector Matrix Tensor

1

$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$

$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

$\begin{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} & \begin{bmatrix} 3 & 2 \end{bmatrix} \\ \begin{bmatrix} 1 & 7 \end{bmatrix} & \begin{bmatrix} 5 & 4 \end{bmatrix} \end{bmatrix}$

NumPy – Vectors, Matrices, and Tensors?

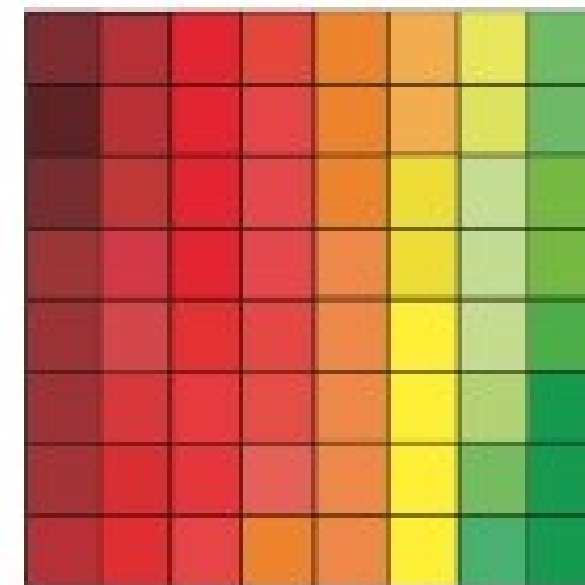
tensor = multidimensional array

vector



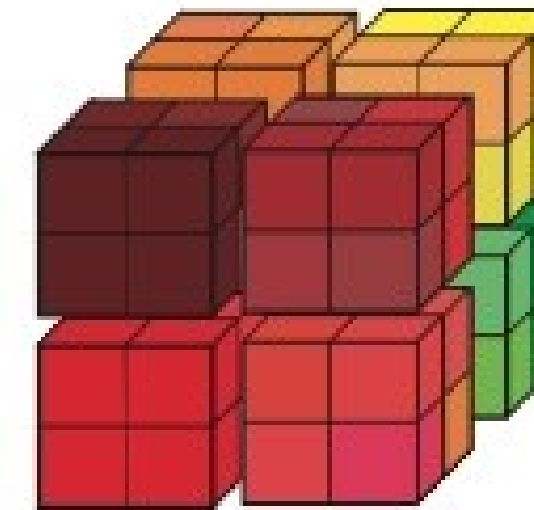
$$\mathbf{v} \in \mathbb{R}^{64}$$

matrix



$$\mathbf{X} \in \mathbb{R}^{8 \times 8}$$

tensor



$$\mathbf{X} \in \mathbb{R}^{4 \times 4 \times 4}$$

Jupyter Notebook

Getting started with the notebook:

File: ***FCD2022-Class4-NumPy-Arrays.ipynb***

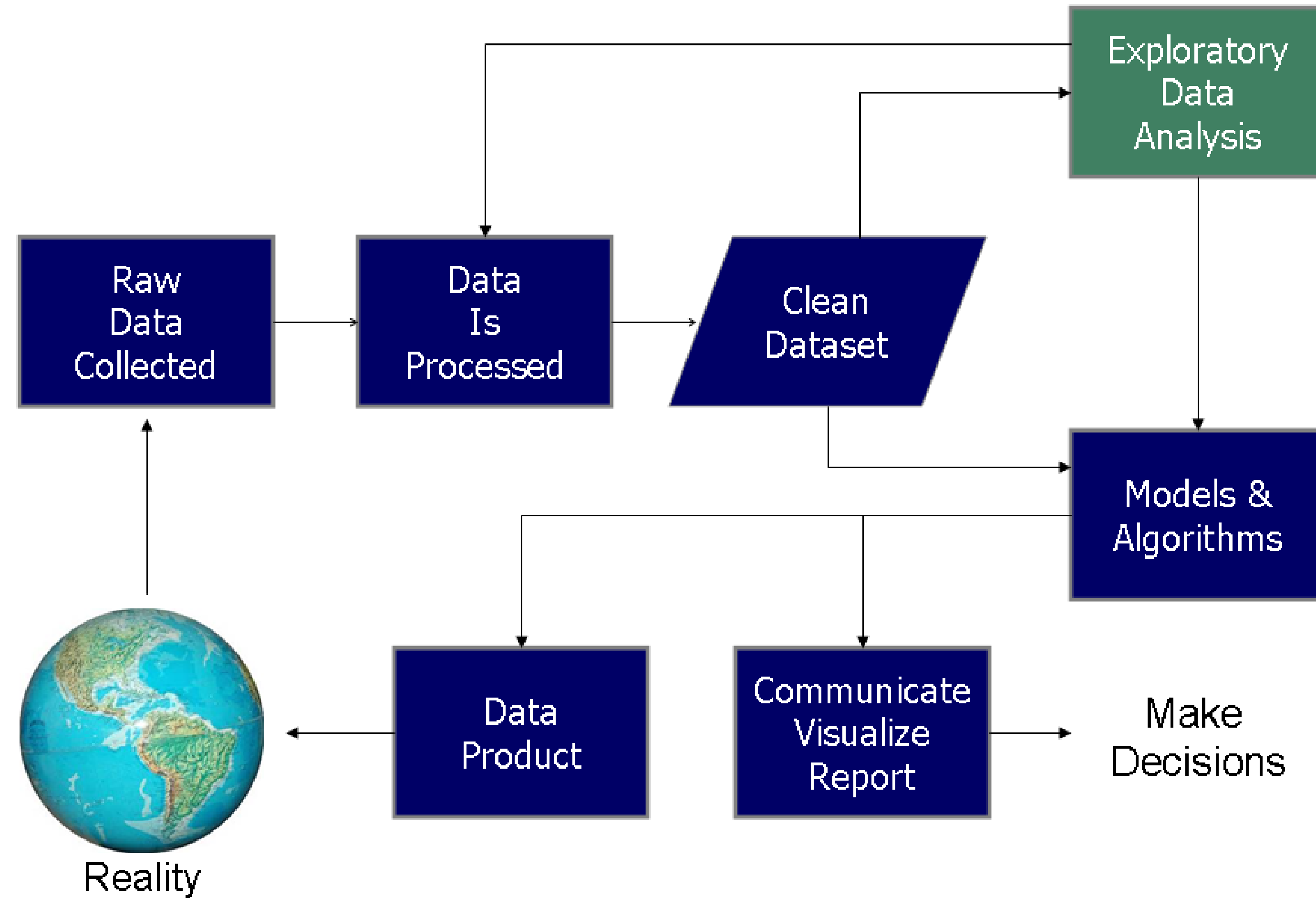
Exploratory Data Analysis (EDA)

- **Exploratory Data Analysis (EDA)** refers to the critical process of performing a kind of "*preliminary investigation*" on data to discover **patterns**, **detect anomalies** (***outliers**, **nulls**, **blanks***) and **generate hypotheses** using descriptive statistics and graphs.
- It is best practice to **interpret the data** and **extract as much information** as possible from it.

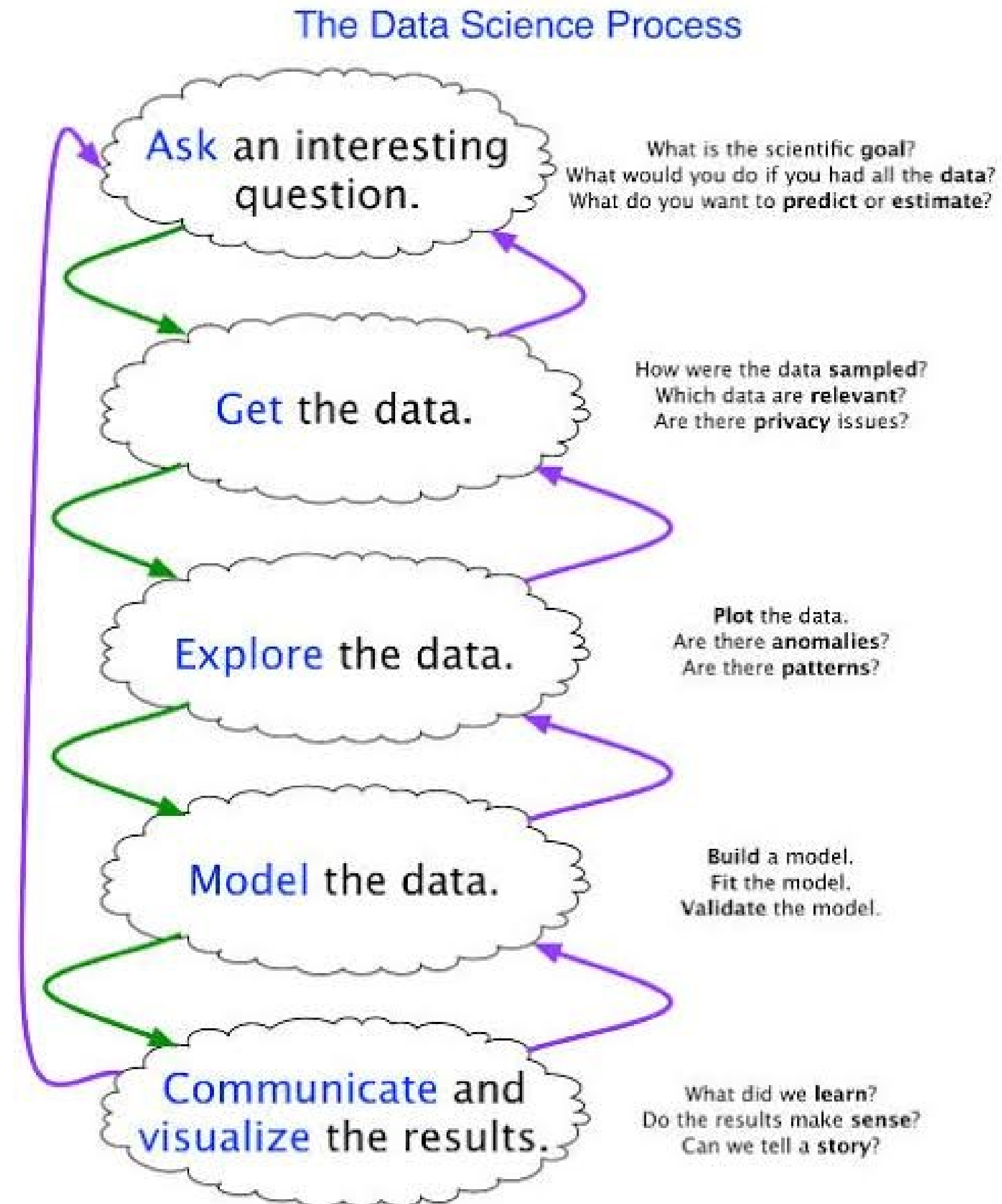
The Importance of EDA

- Identify the most important **variables/features** in your data set.
- Generate one (or several) **hypotheses (questions)** or **check assumptions** related to the dataset.
- Check **data quality** for further **processing and cleaning**.
- Provide **data-driven** insights to business stakeholders.
- Verify existing **expected relationships and unexpected relationships** in the data (through visualization).

Data Science Process & EDA



Data Science Process & EDA



Joe Blitzstein and Hanspeter Pfister, created for the Harvard data science course <http://cs109.org/>.

EDA vs. CDA (Confirmatory Data Analysis)

	Exploratory Data Analysis (EDA)	Confirmatory Data Analysis (CDA)
Reasoning Type	Inductive	Deductive
Goal	Pattern Recognition and Hypothesis generation	Estimation, Modeling, Hypothesis testing
Applied Data	Observation Data (data collected without well-defined hypothesis)	Experimental data (data collected through formally designed experiments)
Techniques	Descriptive Statistics, Data Visualization, Clustering Analysis, Process Mining...	Traditional statistical techniques of inference, significance, and confidence
Advantages	• No assumptions required • Promotes deeper understanding of the data	• Precise • Well-established theory and methods
Disadvantages	• No conclusive answers • Difficult to avoid bias produced by overfitting	• Required unrealistic assumptions • Difficult to notice unexpected results

EDA vs. CDA (Confirmatory Data Analysis)

- We can think of **Confirmatory Data Analysis (CDA)** as if it were the phase of a **trial**:
 - A prosecution / defense lawyer can work all he wants to put up his own ideas based on the material at his disposal, but he won't be able to conclude anything if he can't prove (very accurately) every piece of evidence he has at his disposal.
- Each piece of information contained inside a dataset must be valid for it to be valid.
- **CDA** focuses on evaluating the data and challenging any assumptions you made during the EDA using classic statistical approaches such as **confidence intervals**, **inference**, and **statistical significance** (e.g., *parametric and non-parametric hypothesis tests*).

Pandas – What is it?

- Pandas – An abbreviation for the words 'Panel Data' and 'Python Data Analysis.'
- Designed primarily for working with **relational** or **tabular / labelled** data in a simple and intuitive way.
- It is very similar to the behavior of an **Excel spreadsheet**, a **SQL database**, or a **CSV table** in this type of data, but it is far **more efficient and easier to use**.
- It is based on the **NumPy** library and is exceptionally fast.
- For all these reasons, it's an excellent library for dealing with **massive amounts of data** – N rows x N columns.
- **Data Wrangling** – Exploration, Cleaning, Manipulation (Reshaping, Filtering, Sorting, Merging, Grouping, Concatenating, Splitting, ...)



Pandas – What is its purpose?

Pandas is appropriate for a **wide range of data types**:

- **Tabular data** containing a variety of column kinds, such as in a SQL database or Excel spreadsheet.
- **Time series data** that has been sorted and data that has not been sorted.
- **Matrix data** (input either homogeneously or heterogeneously) with row and column labels.
- **Any additional type of observational / statistical dataset**, without the need of including only labelled data.

Pandas' data structures are:

- **Series**: A 1D vector, like a spreadsheet column.
- **DataFrame**: A 2D array, like a whole spreadsheet.



Pandas – Why?

Why is Pandas so **widely used in data science**?

- Pandas works with **various data science libraries**.
- Because Pandas is built **on top of the NumPy library**, numerous **NumPy structures are used** or replicated, making **huge dataset exploration extremely fast!**
- Pandas' data is frequently used as **input to Matplotlib visualization** routines, **SciPy statistical analysis**, **Scikit-Learn machine learning methods**, and other libraries.



Pandas – Data Structures

Series

- A series in Pandas is **a single-dimensional** (1D - vector) **indexed and labelled array** that can hold **any type of data**.
- When labels are not explicitly specified, they take the **value of the associated indexes**:

index		values
A	→	5
B	→	6
C	→	12
D	→	-5
E	→	6.7

```
pd.Series([1, 2, 2, np.nan], index=['p', 'q', 'r', 's'])
```

↓

		Data
Index	p	1.0
	q	2.0
	r	2.0
	s	NaN

dtype: float64

Series

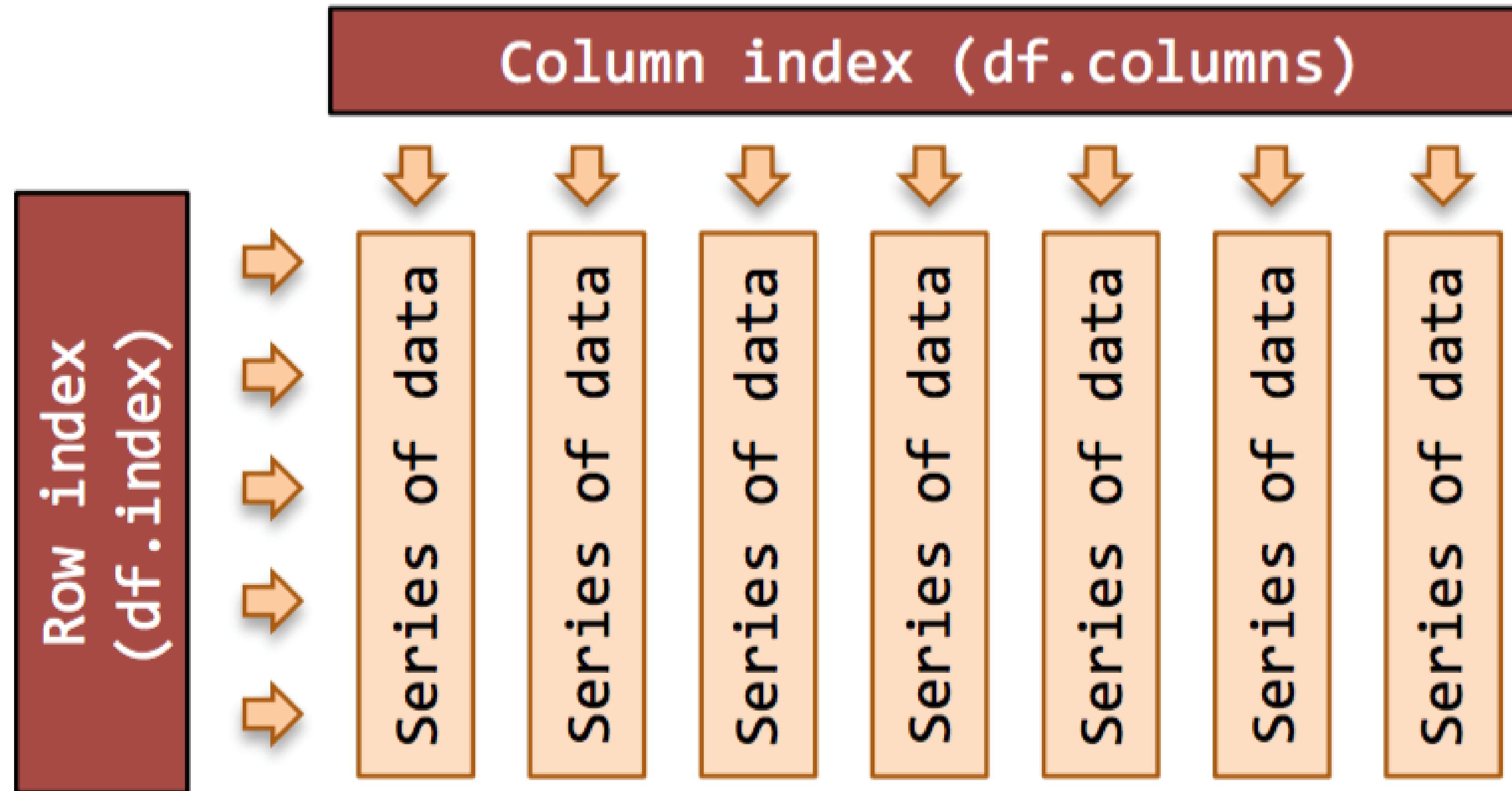
© w3resource.com

Pandas – Data Structures

DataFrame

- In Pandas, a DataFrame is an **indexed and labelled array** of **two dimensions** (2D – matrix) that, like a Series, may **hold any type of data**.
 - Dictionaries containing Lists, Series, NumPy Arrays (*ndarray*), various DataFrames, and so on...
- **Row x column layout** (just like a table in Excel, in a SQL database, etc. - only in format, not in functionalities ☺).
- They also feature **labels** for the **rows (index)** and **columns (columns)**.
- When labels are not explicitly specified, they take the **value of the corresponding indexes**.

Pandas – Series vs. DataFrame



Pandas – Series vs. DataFrame

Series

	apples
0	3
1	2
2	0
3	1

+

Series

	oranges
0	0
1	3
2	7
3	2

=

DataFrame

	apples	oranges
0	3	0
1	2	3
2	0	7
3	1	2

Pandas – Series vs. DataFrame

The diagram illustrates a Pandas DataFrame with the following structure and annotations:

- Column names:** Name, Team, Number, Position, Age, Height, Weight, College, Salary.
- Index labels:** 0, 1, 2, 3, 4, 5, 6.
- Annotations:**
 - Columns axis=1:** Points to the column headers.
 - Index label:** Points to the index values (0-6).
 - Index axis=0:** Points to the index values (0-6).
 - Missing value:** Points to the 'NaN' value in the 'Number' column for index 3.
 - Data:** Points to the numerical values in the 'Age' and 'Weight' columns for index 3.

	Name	Team	Number	Position	Age	Height	Weight	College	Salary
0	Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
1	John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston University	NaN
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0	6-8	235.0	LSU	1170960.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0	6-2	190.0	Louisville	1824360.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN	6-9	260.0	Ohio State	2569260.0
6	Evan Turner	Boston Celtics	11.0	SG	27.0	6-7	220.0	Ohio State	3425510.0

Jupyter Notebook

Getting started with the notebook:

File: ***FCD2022-Class4-DataLoading-Pandas.ipynb***